SIMATS ENGINEERING
(SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Details	Information
Course Code	DSA03
Course Title	Natural Language Processing
CO Assessed	CO2 – Manipulation
Weightage	30% of CIA
Student Name	Ramavath Prashanth Naik
Register Number	192472272
Section / Batch	CSE-AI
Date	20-10-2026

ASSESSMENT TOOL 3 – ANSWERS

Topic: Morphological Error Analysis, Finite-State Parsing, Stemming, and NLP Preprocessing

Question 1 – Morphological Analysis Error in Biomedical Dataset

Aim: To investigate how applying Porter stemming directly to biomedical words can remove useful derivational information and affect search retrieval.

1. Preprocessing and sample experiment

A biomedical corpus such as PubMed 20k RCT can be loaded, text can be lowercased and tokenized, and the Porter Stemmer can be applied to each token. The short example below demonstrates the procedure on the target words; it is not a measured result from the full PubMed dataset.

```
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize

stemmer = PorterStemmer()
words = ["infection", "infectious", "infected", "infect"]

for word in words:
    print(word, "->", stemmer.stem(word))
```

2. Error analysis

Original word	Morphological information	Stemming concern	Impact on search
infection	infect + -ion; noun-forming derivation	Stemmer may reduce it to a form such as 'infect'.	The noun concept may be merged with the verb base.
infectious	infect + -ious; adjective-forming derivation	Derivational meaning 'causing infection' can be lost.	A clinically meaningful adjective distinction may disappear.
infected	infect + -ed; commonly inflectional past/participle	Reduction is often useful for matching word forms.	Can improve recall when users search related forms.
infect	Base form / verb	This is the root form used for comparison.	Provides a common search concept but not full morphology.

3. Inflectional versus derivational morphology

Inflectional morphology changes grammatical form without normally creating a new dictionary word or changing its word class (for example, infect → infected). Derivational morphology can create a related word with a different word class or meaning (for example, infect → infection or infectious). Stemming does not reliably distinguish these functions; therefore, aggressive stemming may merge medically distinct terms.

4. Recommended correction

Use tokenization and normalization consistently, then compare at least three retrieval pipelines: no stemming, Porter stemming, and lemmatization or a domain-aware biomedical normalizer. Evaluate retrieval using precision, recall, or F1 on a labelled query set. Preserve original terms or derivational features for clinical search, rather than relying only on stems.

Conclusion: Porter stemming may improve recall for inflectional variants but can lose derivational distinctions. Biomedical search should evaluate stemming against lemmatization and domain-specific normalization.

Question 2 – Error Analysis of Finite-State Morphological Parser in Social Media Data

Aim: To identify why a simple parser fails on words containing multiple prefixes and suffixes and propose corrected finite-state transitions.

1. Example error cases

Word	Possible analysis	Likely parser failure
happiest	happy + -est (y → i spelling change)	Fails to handle spelling alternation before the superlative suffix.
unbelievable	un- + believe + -able	Recognizes only one affix or fails to process a prefix and suffix together.
running	run + -ing (consonant doubling)	Does not handle consonant doubling or gerund suffix.
reordering	re- + order + -ing	Cannot chain a prefix and suffix around the root.
smartphones	smartphone + -s	May fail to identify a compound base followed by plural -s.
unreadable	un- + read + -able	May not permit both prefix and derivational suffix.

2. Why the parser fails

A parser with only one fixed transition or a single-affix rule cannot represent multiple morphemes. It may also lack spelling rules, a root lexicon, compound handling, and constraints that prevent invalid affix sequences. Social-media text adds misspellings and informal forms, increasing out-of-vocabulary cases.

3. Proposed finite-state design

State / transition	Purpose	Example
Start → optional PREFIX	Recognize prefixes such as un- and re-. Prefix may be absent.	un- / re-
PREFIX → ROOT	Match a valid root from a lexicon or root inventory.	believe, order, read
ROOT → optional DERIV_SUFFIX	Allow derivational suffixes such as -able.	read + -able
ROOT/DERIV_SUFFIX → INFLECTION	Allow inflectional endings such as -s, -ing, -est where valid.	smartphone + -s
Spelling-rule layer	Apply constrained alternations such as happy → happiest and run → running.	y → i; consonant doubling
Accepting state	Accept only sequences permitted by the lexicon and morphology rules.	un + read + able

The finite-state transducer should allow zero or one valid prefix, a root, optional derivational suffixes, and compatible inflectional suffixes. Actual allowed sequences must be constrained by a lexicon and spelling rules; the parser should not accept every arbitrary combination.

4. Evaluation plan and complexity

Create a manually annotated sample from Sentiment140, split it into development and test sets, and score each word as correct or incorrect against gold morphological analyses. Report accuracy before and after correction. No numeric accuracy is provided here because the dataset has not been executed in this report. For a deterministic finite-state transducer, processing a word is typically linear in input length for a fixed automaton; additional lexical lookup and ambiguity handling can add cost. Multiple possible analyses may increase the number of paths that need to be resolved.

Conclusion: Explicit states for prefixes, roots, derivational suffixes, and inflectional suffixes, combined with spelling rules and lexical constraints, improve coverage of multi-affix words.

Question 6 – Error Analysis of a Finite-State Morphological Parser

Aim: To correct a parser that incorrectly removes characters when identifying plural nouns.

1. Identify the logical errors

The original condition checks whether a word ends in 's' but returns word[:-2]. This removes two characters from regular plurals such as cars, producing 'ca' rather than 'car'. It also fails for -es and -ies patterns and does not account for irregular plurals such as children.

2. Corrected rule-based parser

```
IRREGULAR = {
    "children": "child",
    "men": "man",
    "women": "woman",
    "mice": "mouse",
    "teeth": "tooth",
    "feet": "foot",
    "people": "person"
}

def parser(word):
    w = word.lower()

    if w in IRREGULAR:
        return IRREGULAR[w], "Plural Noun (irregular)"
    if w.endswith("ies") and len(w) > 3:
        return w[:-3] + "y", "Plural Noun (-ies)"
    if w.endswith(("ches", "shes", "xes", "zes", "sses")):
        return w[:-2], "Plural Noun (-es)"
    if w.endswith("s") and not w.endswith("ss") and len(w) > 1:
        return w[:-1], "Plural Noun (regular)"
    return w, "Singular / unrecognized"

words = ["cars", "boxes", "cities", "children", "car", "class"]
for word in words:
    print(word, "->", parser(word))
```

3. Illustrative output

Input	Expected base	Parser label
cars	car	Plural Noun (regular)
boxes	box	Plural Noun (-es)
cities	city	Plural Noun (-ies)
children	child	Plural Noun (irregular)
car	car	Singular / unrecognized
class	class	Singular / unrecognized

The output above describes the expected behavior of the corrected rules. The rule-based approach is still incomplete: some nouns have unchanged singular/plural forms (for example, sheep), and exceptions may not follow the listed patterns. A lexical database such as WordNet can provide lemmas for broader coverage.

4. Limitations

Rule-based parsing is transparent and fast, but it requires manually maintained rules and exception lists. It can misclassify unknown words, compounds, spelling variants, and nouns with irregular or unchanged plural forms. For robust use, combine rules with a lexicon and report performance on annotated test examples.

Conclusion: Correct suffix handling and an irregular-word dictionary fix the main slicing error and improve plural normalization, but a rule-based parser cannot cover all English nouns.

Question 7 – Error Analysis of Morphological Feature Extraction

Aim: To fix a text-preprocessing pipeline that stems vocabulary only after CountVectorizer has already built the feature matrix.

1. Identify the preprocessing error

The original program calls CountVectorizer before stemming. As a result, variants such as running, runs, and runners are created as separate vocabulary features. Stemming the displayed feature names afterward only changes the printed list; it does not merge the corresponding columns in the already-created matrix X.

2. Corrected pipeline

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "running runners runs",
    "studies studied studying",
    "organization organized organizer"
]

def stem_tokenizer(text):
    # Simple whitespace tokenizer for this demonstration.
    # For general text, use a tokenizer that handles punctuation properly.
    return [stemmer.stem(token.lower()) for token in text.split()]

vectorizer = CountVectorizer(
    tokenizer=stem_tokenizer,
    token_pattern=None,
    lowercase=False
)

X = vectorizer.fit_transform(documents)
features = vectorizer.get_feature_names_out()

print("Vocabulary:", list(features))
print("Vocabulary size after stemming:", len(features))
print("Document-term matrix shape:", X.shape)
```

3. Comparison and evaluation

Measure	Before correction	After correction
Pipeline order	Vectorize first, stem names later	Stem tokens before vocabulary construction
Vocabulary	Inflectional variants remain separate columns	Variants with the same stem can share a feature
Feature matrix	Columns are not merged by editing printed names	CountVectorizer builds columns from normalized tokens
Classification accuracy	Must be measured on a held-out set	Must be measured using the same split and model
Processing time	Measure full vectorization pipeline	Measure tokenization + stemming + vectorization

A valid experiment should compare the original and corrected pipelines on the same real-world corpus, using the same train/test split and classifier. Record vocabulary size, accuracy (or F1), and elapsed time. Exact values are not reported here because the 20 Newsgroups or other dataset was not run in this environment.

4. Interpretation

Stemming before vectorization can reduce redundant features, reduce vocabulary size, and help the classifier generalize across related word forms. However, stemming can also merge words with different meanings, so the effect on accuracy must be measured rather than assumed. For broader text, use a robust tokenizer and consider lemmatization.

Conclusion: Normalization must happen before feature extraction so the document-term matrix is built from normalized tokens, not merely from a modified display list.

Overall Conclusion

This report demonstrates four error-analysis tasks: the risks of stemming biomedical derivations, the need for multi-affix finite-state parsing, correction of plural suffix handling, and correct ordering of stemming before feature extraction. The main lesson is to preserve morphological information where it affects meaning and to validate all preprocessing changes with annotated data and measurable evaluation results.

References / Tools

NLTK PorterStemmer; scikit-learn CountVectorizer; PubMed 20k RCT Dataset (suggested by the assessment); Sentiment140 Dataset (suggested by the assessment); WordNet lexical database (suggested for parser validation). Dataset-specific metrics should be filled in after actual execution.